

1. Estructura de una clase en POO

```
public class Sala {
    private int numero;
    private String pelicula;

    public Sala(int numero, String pelicula) {
        this.numero = numero;
        this.pelicula = pelicula;
    }

    public int getNumero() { return numero; }
    public void setNumero(int numero) { this.numero = numero; }

    @Override
    public String toString() {
        return "Sala " + numero + ": " + pelicula;
    }
}
```

Qué hace:

- Define atributos
 - Constructor para crear objetos
 - Getters/Setters para acceder
 - toString para imprimir
-

2. Crear objetos e introducirlos en un ArrayList

```
ArrayList<Sala> salas = new ArrayList<>();

salas.add(new Sala(1, "Avatar"));
salas.add(new Sala(2, "Oppenheimer"));
```

Qué hace:

Guarda objetos en una lista dinámica.

3. Recorrer un ArrayList (for-each)

```
for (Sala s : salas) {
    System.out.println(s);
}
```

Qué hace:

Imprime cada objeto usando su toString().

4. Buscar un objeto dentro de un ArrayList

```
public Sala buscarSala(int num) {
    for (Sala s : salas) {
        if (s.getNumero() == num) {
            return s;
        }
    }
    return null;
}
```

Qué hace:

Devuelve la sala cuyo número coincide.

5. Matriz de asientos (cine)

```
char[][] asientos = new char[6][10];

for (int i = 0; i < 6; i++) {
    for (int j = 0; j < 10; j++) {
        asientos[i][j] = '.';
    }
}
```

Qué hace:

Crea una sala con asientos libres (.).

6. Comprobar asiento disponible

```
public boolean asientoLibre(int f, int c) {
    return asientos[f][c] == '.';
}
```

Qué hace:

Devuelve true si el asiento está libre.

7. Vender entrada

```
public void venderEntrada(int f, int c) {
    if (asientoLibre(f, c)) {
        asientos[f][c] = 'X';
        recaudacionSala += PRECIO;
        recaudacionTotal += PRECIO;
    }
}
```

Qué hace:

Marca asiento como ocupado y suma dinero.

8. HashMap (clave → valor)

```
HashMap<String, Integer> tiempos = new HashMap<>();

tiempos.put("Faura", 950);
tiempos.put("Gilet", 1000);

int t = tiempos.get("Faura");
```

Qué hace:

Guarda poblaciones con su tiempo.

9. Recorrer HashMap

```
for (Map.Entry<String, Integer> e : tiempos.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

Qué hace:

Imprime clave y valor.

10. HashSet (sin duplicados)

```
HashSet<String> visitadas = new HashSet<>();

visitadas.add("Faura");
visitadas.add("Faura"); // no se duplica
```

Qué hace:

Guarda poblaciones visitadas sin repetir.

11. Crear ruta (ArrayList + HashMap)

```
ArrayList<String> ruta = new ArrayList<>();

for (String p : tiempos.keySet()) {
    if (!visitadas.contains(p)) {
        ruta.add(p);
    }
}
```

Qué hace:

Añade solo poblaciones NO visitadas.

12. Ordenar ruta por tiempo (Comparator)

```
Collections.sort(ruta, (a, b) -> tiempos.get(a) - tiempos.get(b));
```

Qué hace:

Ordena la ruta de menor a mayor tiempo.

13. Calcular tiempo total

```
int total = 0;
for (String p : ruta) {
    total += tiempos.get(p);
}
```

Qué hace:

Suma los milisegundos de la ruta.

14. Mostrar ruta formateada

```
for (int i = 0; i < ruta.size(); i++) {
    System.out.print "[" + ruta.get(i) + "];";
    if (i < ruta.size() - 1) System.out.print "->";
}
```

Qué hace:

Imprime: [Faura]->[Gilet]->[Canet]

15. Recaudación total (static)

```
private static double recaudacionTotal = 0;

public static double getRecaudacionTotal() {
    return recaudacionTotal;
}
```

Qué hace:

Variable compartida por TODAS las salas.

16. Menú básico con switch

```
char op;
do {
    mostrarMenu();
    op = sc.nextLine().toUpperCase().charAt(0);

    switch (op) {
        case 'A': crearSala(); break;
        case 'B': verCartelera(); break;
    }
}
```

```
        case 'C': verSala(); break;
        case 'D': venderEntrada(); break;
        case 'E': verRecaudacionSala(); break;
        case 'F': verRecaudacionTotal(); break;
        case 'G': System.out.println("Saliendo..."); break;
        default: System.out.println("Opción no válida");
    }
} while (op != 'G');
```

Qué hace:

Controla el programa hasta que el usuario pulse G.

17. Limpieza del buffer (muy importante)

```
int num = sc.nextInt();
sc.nextLine(); // limpiar salto de línea
```

Qué hace:

Evita que nextLine() lea vacío.

18. toString en objetos para imprimir fácil

```
@Override
public String toString() {
    return "Sala " + num + " - " + pelicula;
}
```

Qué hace:

Permite imprimir objetos directamente.

19. Validar índices de matriz

```
if (fila < 0 || fila >= FILAS || col < 0 || col >= COLUMNAS) {
    System.out.println("Fuera de rango");
}
```

Qué hace:

Evita errores al acceder a la matriz.

20. Estructura típica de un método de búsqueda

```
public Sala buscarSala(int num) {
    for (Sala s : salas) {
        if (s.getNumSalas() == num) return s;
    }
    return null;
}
```

Qué hace:

Devuelve la sala o null si no existe.

```
1 package Entregable;
2 import java.util.Scanner;
3 import java.util.ArrayList;
4
5
6 public class MainSala {
7
8     static ArrayList<Sala> salas = new ArrayList<>(); 6 usages
9     static Scanner sc = new Scanner(System.in); 9 usages
10
11     public static void main(String[] args) {
12
13         salas.add(new Sala(1, "Deadpool", "17:00"));
14         salas.add(new Sala(2, "Oppenheimer", "20:00"));
15         salas.add(new Sala(3, "Avatar 3", "23:00"));
16
17         char opcion;
```

```
1 package Entregable;
2
3 public class Sala { 12 usages
4     char[][] Asientos; 6 usages
5     private static final int Filas = 6; 5 usages
6     private static final int Columnas = 10; 6 usages
7     private static double PEntrada = 5.99; 2 usages
8     private int numSalas; 5 usages
9     private String Pelicula; 5 usages
10    private String horaProyeccion; 5 usages
11    private double recaudacionSala; 4 usages
12    private static double recaudacionTotal = 0; 2 usages
13
14    public Sala() { no usages
15        this.numSalas = 0;
16        this.Pelicula = "";
17        this.horaProyeccion = "";
18        this.recaudacionSala = 0;
19        inicializarAsientos();
20    }
21    public Sala(int numSalas, String Pelicula, String horaProyeccion){ 4 usages
22        this.numSalas = numSalas;
23        this.Pelicula = Pelicula;
24        this.horaProyeccion = horaProyeccion;
25        this.recaudacionSala = 0;
26        inicializarAsientos();
27    }
28    private void inicializarAsientos(){ 2 usages
29        Asientos = new char[Filas][Columnas];
30        for (int i = 0; i < Filas; i++) {
31            for (int j = 0; j < Columnas; j++) {
32                Asientos [i][j] = '.';
33            }
34        }
35    }
36    public void mostrarSala(){ 1 usage
37        System.out.print(" ");
38        for (int j = 0; j < Columnas; j++) {
39            System.out.print(j + " ");
40        }
41        System.out.println();
```

```
public static double getRecaudacionTotal() { return recaudacionTotal; }
@Override
public String toString(){
    return "[Sala " + numSalas + "]" + Pelicula + " (" + horaProyeccion + ") - Asientos Libres: " + getAsientosDisponibles();
}

public int getNumSalas() { return numSalas; }
public void setNumSalas(int numSalas) { this.numSalas = numSalas; }
public String getPelicula() { return Pelicula; }
public void setPelicula(String Pelicula) { this.Pelicula = Pelicula; }
public String getHoraProyeccion() { return horaProyeccion; }
public void setHoraProyeccion(String horaProyeccion) { this.horaProyeccion = horaProyeccion; }
```